This is a mixture of C++/pseudocode outlining the basic functioning of the polygon pipeline. Make changes as needed.

```
int poly_pipeline(attr_point p, int end_flag)
{
  pointh geom, norm;
  pointh dev;

  const int MAX_VERTEX_LIST_SIZE = 50;
  static attr_point  vertex_list[MAX_VERTEX_LIST_SIZE];
  static attr_point  clipped_list[MAX_VERTEX_LIST_SIZE];
  static int n_vertex = 0;

  int i;

  // Run geometry through current transform
  geom[0] = p.coord[0];
  geom[1] = p.coord[1];
  geom[2] = p.coord[2];
  geom[3] = p.coord[3];

  geom = cur_xform * geom;

  // Run through world to clip
  geom = world_to_clip * geom;

  p.coord[0] = geom[0];
  p.coord[1] = geom[1];
  p.coord[2] = geom[2];
  p.coord[3] = geom[3];

  // Store in vertex list
  if( n_vertex == MAX_VERTEX_LIST_SIZE)
    return -1;  // Overflow

  vertex_list[n_vertex] = p;
  n_vertex++;

  if(end_flag == MOVE)  // Move along to the next vertex
    return 0;

  if((n_vertex = poly_clip(n_vertex, vertex_list, clipped_list)))
    {
      // There's something left! --- Let's draw it

      // Pre process vertex list
      for(i = 0; i < n_vertex; i++)
        {
          // Convert geometry to device coordinates
          dev[0] = clipped_list[i].coord[0];
          dev[1] = clipped_list[i].coord[1];
          dev[2] = clipped_list[i].coord[2];
          dev[3] = clipped_list[i].coord[3];
          dev = clip_to_device * dev;

          clipped_list[i].coord[0] = dev[0];
          clipped_list[i].coord[1] = dev[1];
          // clipped_list[i].coord[2] = dev[2];  // Superfluous
          // clipped_list[i].coord[3] = dev[3];

          // Divide geometry by w.
          clipped_list[i].coord[0] /= clipped_list[i].coord[3];
          clipped_list[i].coord[1] /= clipped_list[i].coord[3];
```

```
          clipped_list[i].coord[2] /= clipped_list[i].coord[3];

        }

      scan_convert(clipped_list, n_vertex);
    }

  // Reset structures for next polygon

  n_vertex = 0;
}
```